Chess Master

Object-Oriented Chess Game with SFML Graphics

Short Project Report

Submitted by: **Moaz Shafie**

Student ID: **25F-0029**

Submitted to: **Hassan Sami**

Date: **14 May 2026**

Course: **Object-Oriented Programming (OOP)**

1. Introduction

Chess Master is a fully playable two-player chess application developed in **C++** using the **SFML (Simple and Fast Multimedia Library)** for graphics and input. The project was built as a practical demonstration of core Object-Oriented Programming principles applied to a real, interactive system.

The game supports two modes: **1 Player vs Computer** (where the computer plays black using random legal-move selection) and **2 Players Local** (hot-seat). All standard chess rules are implemented, including castling, en passant, pawn promotion, check detection, checkmate, and stalemate. The graphical interface renders an 8×8 board with textured piece sprites, a highlighted selected square, and a side panel showing game status — all driven entirely by mouse input.

The codebase is split across three files: **Header.h** (declarations), **Implementation.cpp** (all logic and rendering), and **Main.cpp** (entry point) — following standard modular C++ practice.

2. Object-Oriented Programming Concepts Used

2.1 Encapsulation

Each class hides its internal data behind a clean public interface. The **Piece** base class stores *color* (char) and *hasMoved* (bool) as protected members, exposing them only through *getters* and *setters*. The **Board** class owns the 8×8 grid of *Piece** pointers privately; outside code interacts only through *Move()*, *IsCheck()*, and similar methods — never touching the grid directly.

2.2 Inheritance

A single abstract **Piece** base class defines the common interface for every chess piece. Six concrete subclasses inherit from it:

Subclass	Symbol	Unique Movement Rule
Pawn	P / p	Forward 1 (or 2 initially), diagonal capture, en passant
Rook	R / r	Horizontal & vertical sliding
Knight	N / n	L-shaped jump; ignores blocking pieces
Bishop	B / b	Diagonal sliding
Queen	Q / q	Combined Rook + Bishop rules (via delegation)
King	K / k	One square any direction; castling

2.3 Polymorphism

The **Piece** base class declares two *pure-virtual* methods:

```
virtual bool IsValidMove(int sx, int sy, int dx, int dy, Piece* Grid[8][8]) = 0;
virtual char GetSymbol() const = 0;
```

The **Board::IsCheck()** and **Board::HasAnyValidMove()** methods call *IsValidMove()* through a *Piece** pointer without any switch-on-type logic — the correct subclass method is dispatched automatically at run time. This is textbook runtime polymorphism and is central to keeping check detection and move generation clean and extensible.

2.4 Abstraction

Complex operations are hidden behind simple interfaces. Callers invoke **Board::Move(sx, sy, dx, dy, Turn)** and receive a bool result; they never need to know about the internal simulation, undo logic, or en passant bookkeeping that happens inside. Similarly, **Game::MakeComputerMove()** encapsulates the entire AI routine behind a single call.

2.5 Composition

The **Game** class contains a **Board** object by value (not inheritance) — a deliberate use of *composition over inheritance*. The **Queen** class reuses movement rules by delegating to temporary **Rook** and **Bishop** instances inside *IsValidMove()*, avoiding code duplication without multiple inheritance.

3. Class Diagram

```
+================================+ +============================+ | <<abstract>> Piece | |
Board | |--------------------------------| |----------------------------| | # color : char | |
- Grid[8][8] : Piece* | | # hasMoved : bool | | - lastMove : struct |
|--------------------------------| owns |----------------------------| | + GetColor() : char |
<-------| + Setup() | | + IsValidMove(...) : bool [PV] | 0..* | + Move(sx,sy,dx,dy) : bool | | +
GetSymbol() : char [PV] | | + IsCheck(char) : bool | +----------+--------------------+ | +
HasAnyValidMove() : bool | | | + PromotePawn(x,y) | ________|_________
+------------+--------------+ / / | \ \ | composition Pawn Rook Knight Bishop | \
+===========+============+ Queen (delegates to Rook+Bishop)| Game | King
|-----------------------| | - B : Board | | - Turn : char | | - vsComputer: bool | | - window :
RWindow | | - textures : map | |-----------------------| | + Start() | | + HandleEvents() | | +
Render() | | + RenderMenu() | | + MakeComputerMove() | | + LoadAssets() |
+========================+
```

Figure 1: UML Class Diagram — Chess Master

4. Screenshots of Output

Screenshot 1: Main Menu Screen

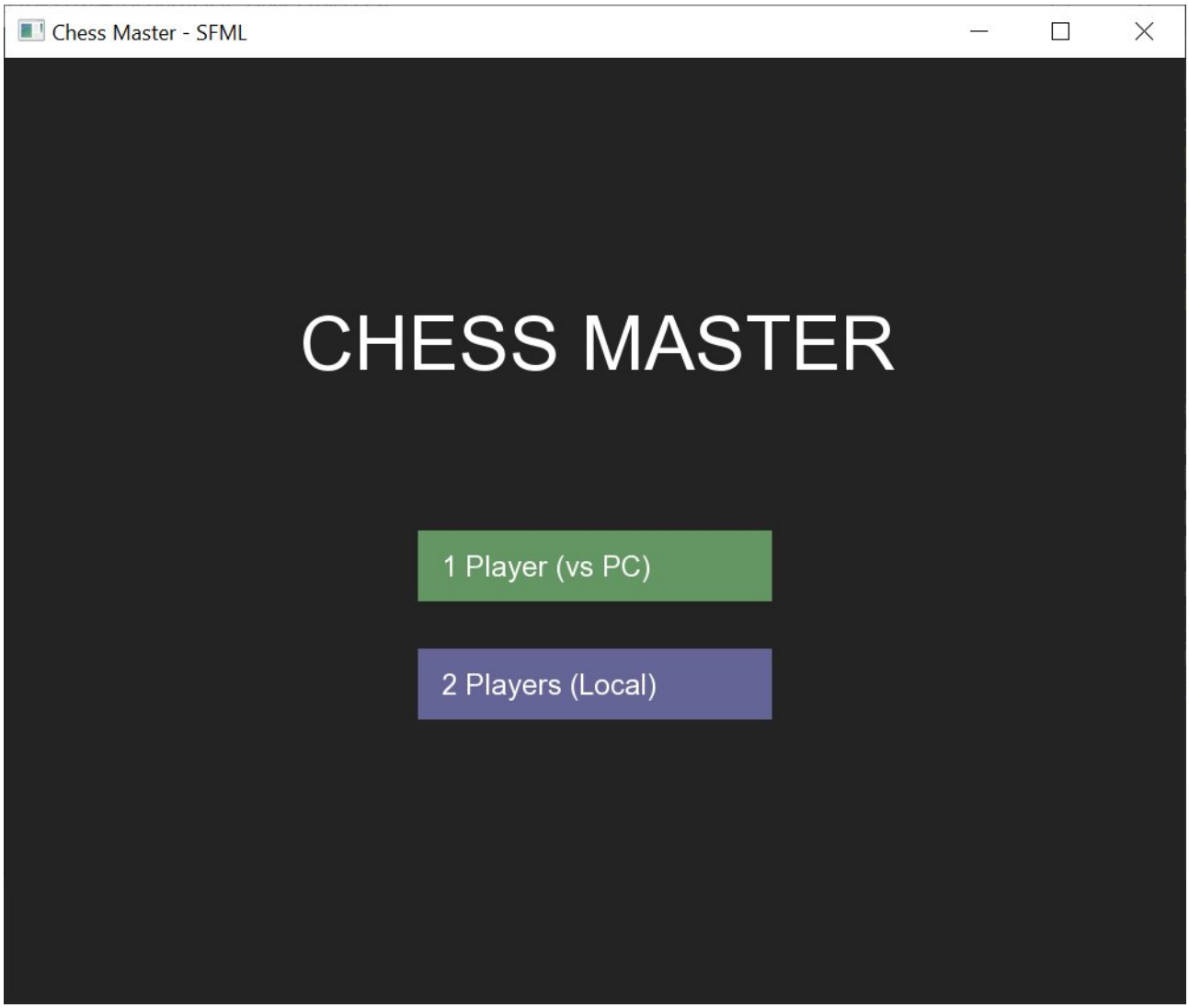


Figure 2: The main menu presents two mode-selection buttons — 1 Player (vs PC) and 2 Players (Local) — on a clean dark-themed background. Clicking either button launches the game in the corresponding mode.

Screenshot 2: In-Game Board (Starting Position)



Figure 3: The 8×8 board at the start of a game in 2-Player mode. All 32 pieces are correctly placed using textured PNG sprites. The right-hand panel displays the game title and current turn indicator (Player 1 (White)'s Turn). The alternating wood-tone cells and high-quality piece artwork demonstrate the SFML rendering pipeline.